

Seminar 5

Problema celui mai scurt drum.

Algoritmul lui Dijkstra

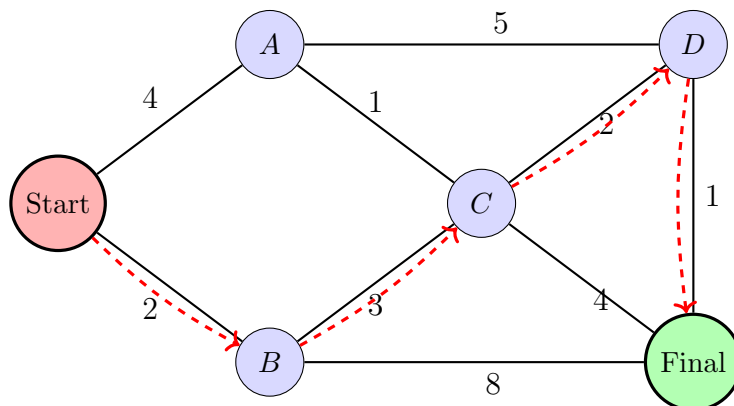
Asist. univ. dr. Mihai Tudor

16 martie 2026

1 Motivație și context

Problema celui mai scurt drum apare în nenumărate aplicații practice: navigație GPS, rețele de calculatoare (rutare pachete), planificare logistică, hărți de transport. Formularea matematică este simplă: **dat un graf ponderat** $G = (V, E, w)$ **și un vârf sursă** s , **găsiți drumul de cost minim de la** s **la fiecare alt vârf**.

Algoritmul lui **Edsger W. Dijkstra** (1959) rezolvă această problemă eficient pentru grafuri cu **ponderi nenegative**. Este unul dintre cei mai eleganți și mai studiați algoritmi din informatică.



Drum optim: $\text{Start} \rightarrow B \rightarrow C \rightarrow D \rightarrow \text{Final}$, $\text{cost} = 2 + 3 + 2 + 1 = 8$

(alternativa $\text{Start} \rightarrow A \rightarrow C \rightarrow D \rightarrow \text{Final}$: $\text{cost} = 4 + 1 + 2 + 1 = 8$, la fel; $\text{Start} \rightarrow B \rightarrow \text{Final}$: $2 + 8 = 10$)

2 Definiții fundamentale

Definiție 2.1. Un **graf ponderat** $G = (V, E, w)$ este un graf în care fiecărei muchii $e \in E$ i se asociază un număr real $w(e)$ numit **pondere** (sau **cost**).

Definiție 2.2. Fie $G = (V, E, w)$ un graf ponderat și $P = (v_0, v_1, \dots, v_k)$ un drum de la v_0 la v_k . **Lungimea** (sau **costul**) drumului P este:

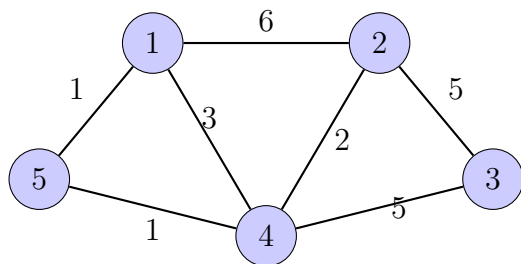
$$w(P) = \sum_{i=0}^{k-1} w(v_i, v_{i+1})$$

Drumul de cost minim (sau **drumul scurt**) de la s la t este drumul P^* cu $w(P^*) = \min_{P:s \rightsquigarrow t} w(P)$.

Definiție 2.3. Fie $\delta(s, v)$ costul drumului scurt de la sursa s la vârful v . Prin convenție, $\delta(s, s) = 0$ și $\delta(s, v) = \infty$ dacă nu există drum de la s la v .

Observație 2.1. De ce avem nevoie de ponderi nenegative?

Dacă există muchii cu pondere negativă, pot apărea **cicluri de cost negativ** (un ciclu al cărui cost total este < 0). Parcurgând un astfel de ciclu de infinit de ori, costul drumului ar tinde la $-\infty$, deci problema nu mai are soluție. Algoritmul lui Dijkstra presupune că $w(e) \geq 0$ pentru orice $e \in E$.

Exemplu 2.1. Reprezentarea unui graf ponderat*Reprezentare vizuală**Matricea de adiacență W* *(0 = lipsă muchie, ∞ = nedefinit)*

$$W = \begin{pmatrix} 0 & 6 & \infty & 3 & 1 \\ 6 & 0 & 5 & 2 & \infty \\ \infty & 5 & 0 & 5 & \infty \\ 3 & 2 & 5 & 0 & 1 \\ 1 & \infty & \infty & 1 & 0 \end{pmatrix}$$

```

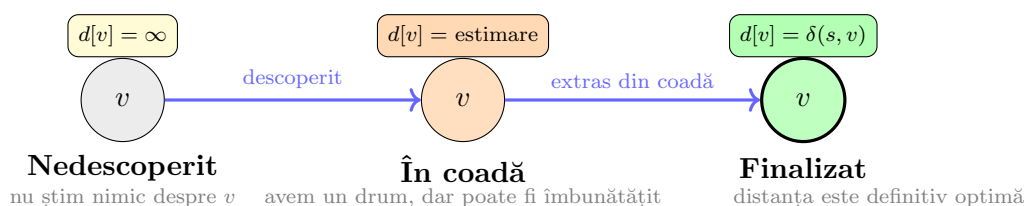
1 # Reprezentare ca dictionar de adiacenta (lista cu costuri)
2 graf = {
3     1: [(2, 6), (4, 3), (5, 1)], # (vecin, cost)
4     2: [(1, 6), (3, 5), (4, 2)],
5     3: [(2, 5), (4, 5)],
6     4: [(1, 3), (2, 2), (3, 5), (5, 1)],
7     5: [(1, 1), (4, 1)]
8 }

```

3 Algoritmul lui Dijkstra

3.1 Ideea fundamentală

Dijkstra pornește dintr-o **sursă** s și “cucerește” vârfurile unul câte unul, în ordinea crescătoare a distanței față de s . La fiecare moment, fiecare vârf se află în una dintre trei stări:



Fie s sursa și $d[v]$ **cel mai bun cost cunoscut** până în momentul curent pentru a ajunge de la s la v . La început nu știm nimic despre drumuri, deci $d[s] = 0$ (sursa este la distanță zero față de ea însăși) și $d[v] = \infty$ pentru orice alt vârf v (nu am descoperit niciun drum spre el).

Pe măsură ce algoritmul avansează, $d[v]$ se poate *îmbunătăți* (micșora) ori de câte ori descoperim un drum mai scurt spre v . Când nu mai există nicio posibilitate de îmbunătățire pentru un vârf, îl declarăm **finalizat** — valoarea $d[v]$ devine definitivă și egală cu distanța optimă $\delta(s, v)$.

Întrebarea cheie este: *în ce ordine finalizăm vârfurile?* Dijkstra răspunde simplu: **întotdeauna finalizăm mai întâi vârful cu cel mai mic $d[v]$ curent**. Motivul este că, dacă toate ponderile sunt ≥ 0 , niciun drum viitor nu poate face $d[v]$ mai mic decât

este el la momentul extragerii — orice drum alternativ ar trebui să treacă printr-un alt vârf cu $d \geq d[v]$, deci ar fi cel puțin la fel de lung.

Odată extras un vârf u , îi examinăm toți vecinii v și verificăm: *drumul prin u spre v este mai scurt decât ce știam?* Această verificare și eventuala actualizare a lui $d[v]$ se numește **relaxarea** muchiei (u, v) .

3.2 Operația de relaxare

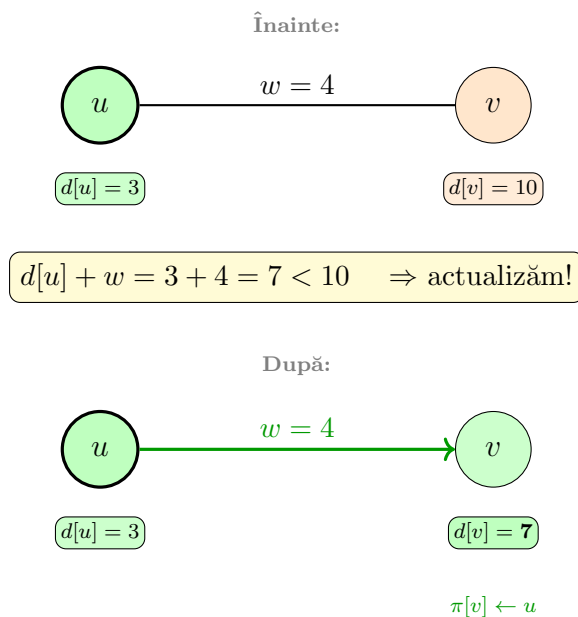
Definiție 3.1. Operația de **relaxare** a muchiei (u, v) cu cost $w(u, v)$:

$$\text{dacă } d[u] + w(u, v) < d[v] \quad \text{atunci} \quad d[v] \leftarrow d[u] + w(u, v), \pi[v] \leftarrow u$$

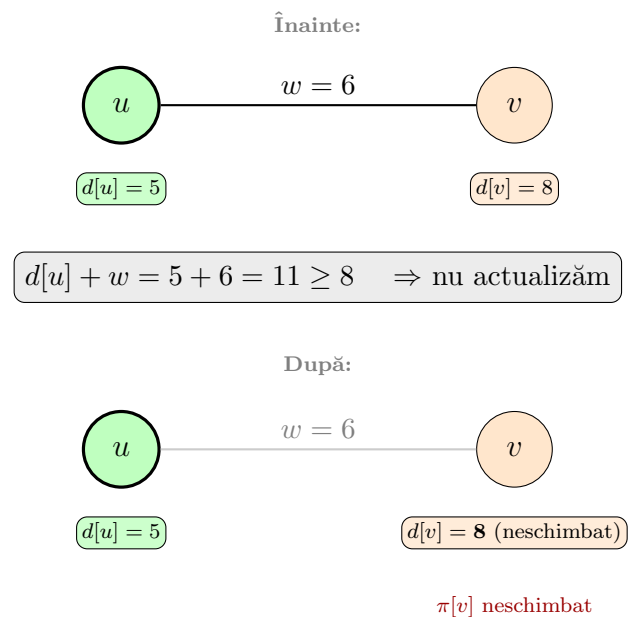
unde $\pi[v]$ este **predecesorul** lui v pe drumul scurt curent (folosit pentru reconstruirea drumului).

Relaxarea se poate imagina astfel: “dacă îl trec pe v prin u , obțin un drum mai scurt decât ce știam până acum?”

Caz 1 — Relaxare reușită



Caz 2 — Relaxare eșuată



3.3 Pașii algoritmului

- (1) **Inițializare:** $d[s] \leftarrow 0$, $d[v] \leftarrow \infty$ pentru $v \neq s$, $\pi[v] \leftarrow \text{None}$ pentru toți v . Introducem s în coada de priorități cu prioritate 0.
- (2) **Cât timp coada nu este goală:**
 - (a) Extrage vârful u cu $d[u]$ minim din coadă.
 - (b) Dacă u a mai fost procesat, ignoră (poate apărea de mai multe ori în heap).
 - (c) Pentru fiecare vecin v al lui u : încearcă relaxarea muchiei (u, v) . Dacă $d[v]$ se îmbunătățește, adaugă v în coadă cu noua prioritate.

(3) **Returnează** vectorii d (distanțe minime) și π (predecesori).

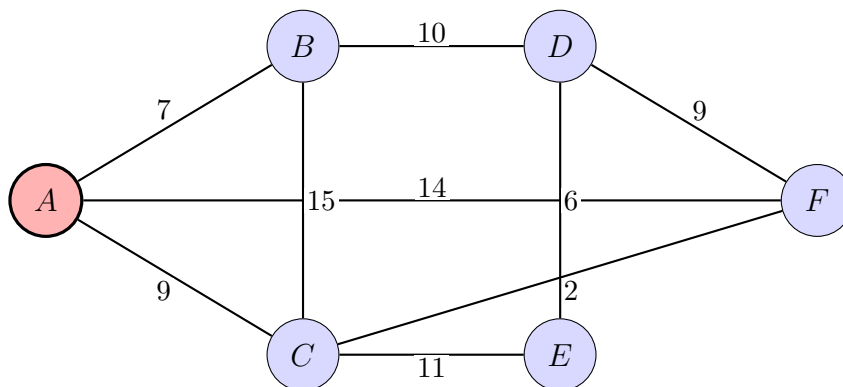
Observație 3.1. De ce este corect? Când extragem u din coadă cu $d[u]$ minim, afirmăm că $d[u]$ este deja optim.

Demonstrație prin contradicție: dacă ar exista un drum mai scurt $s \rightsquigarrow \dots \rightsquigarrow x \rightsquigarrow \dots \rightsquigarrow u$, atunci x (primul vârf nefinalizat pe acel drum) ar trebui să aibă $d[x] \leq d[u]$. Dar noi am ales u ca minimul din coadă — contradicție. Argumentul funcționează **numai** pentru ponderi ≥ 0 : o pondere negativă ar putea face un drum “mai scurt” după ce u a fost finalizat.

4 Exemplu pas cu pas — urmărire vizuală completă

Exemplu 4.1. Urmărirea algoritmului Dijkstra pe un graf cu 6 vârfuri, pornind din A .

Graful de lucru (vârfurile și ponderile muchiilor):



Sursă: A , distanțe inițiale: toate ∞ (exceptând $d[A] = 0$)

Legendă culori în diagramele de mai jos:



Sursă



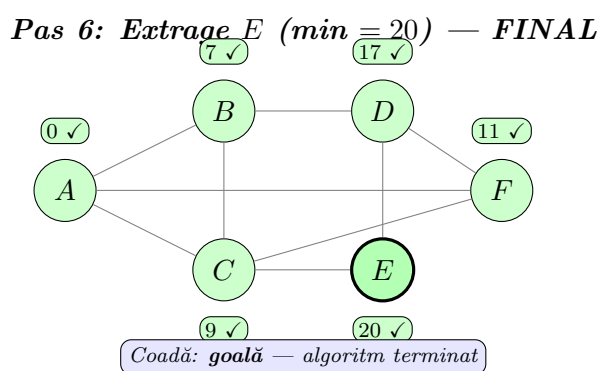
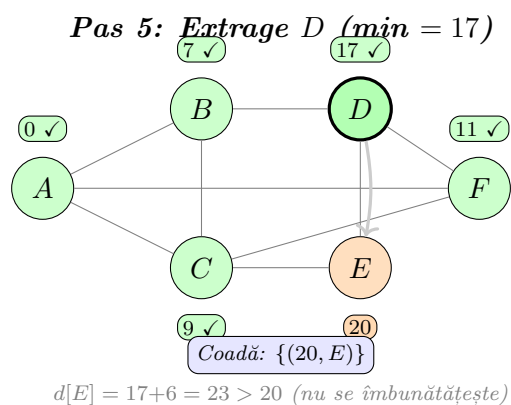
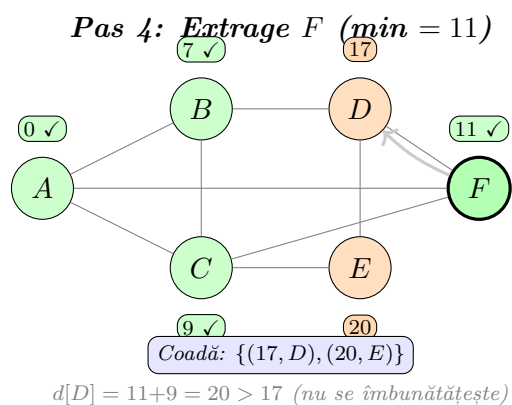
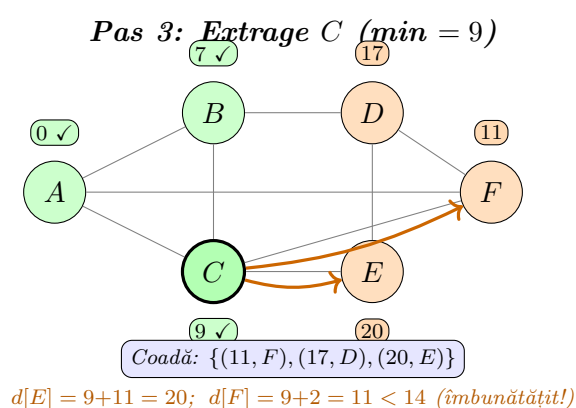
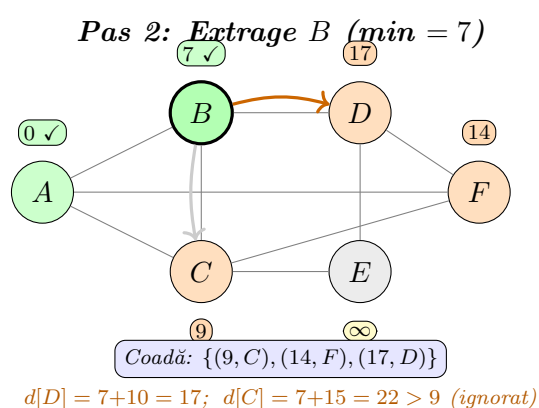
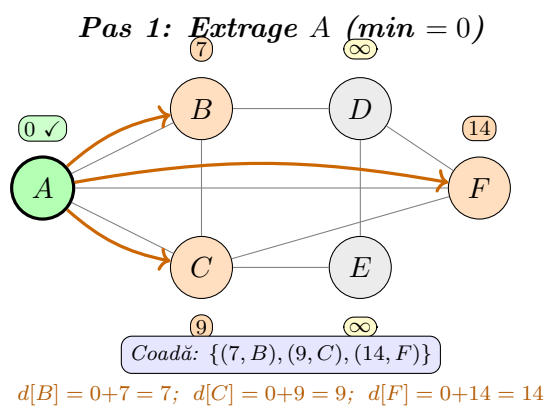
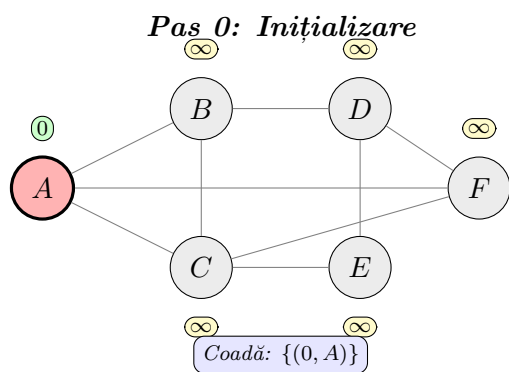
Nedescoperit (∞) În coadă (estimare)



Curent (extras)



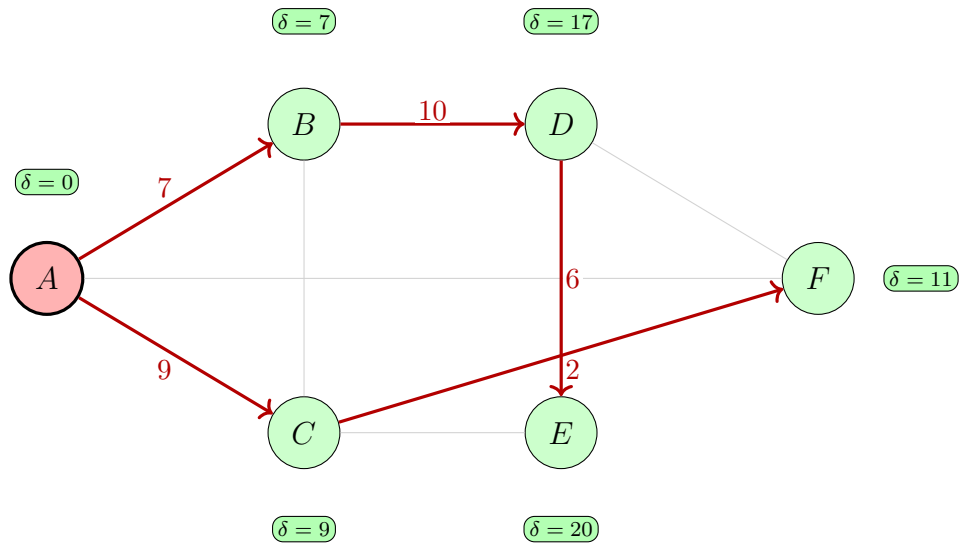
Finalizat



Tabel de urmărire complet (galben = vârf selectat curent, verde = finalizat):

<i>Pas</i>	<i>Extras</i>	$d[A]$	$d[B]$	$d[C]$	$d[D]$	$d[E]$	$d[F]$	<i>Relaxări efectuate</i>
0	—	0	∞	∞	∞	∞	∞	Inițializare
1	A	0	7	9	∞	∞	14	$d[B] = 7$; $d[C] = 9$; $d[F] = 14$
2	B	0	7	9	17	∞	14	$d[D] = 7+10 = 17$; $d[C] = 7+15 = 22 > 9$ ignorat
3	C	0	7	9	17	20	11	$d[E] = 9+11 = 20$; $d[F] = 9+2 = 11 < 14$
4	F	0	7	9	17	20	11	$d[D] = 11+9 = 20 > 17$ ignorat
5	D	0	7	9	17	20	11	$d[E] = 17+6 = 23 > 20$ ignorat
6	E	0	7	9	17	20	11	Coadă goală — terminat

Arborele drumurilor scurte din A (reconstruit din predecesorii π):



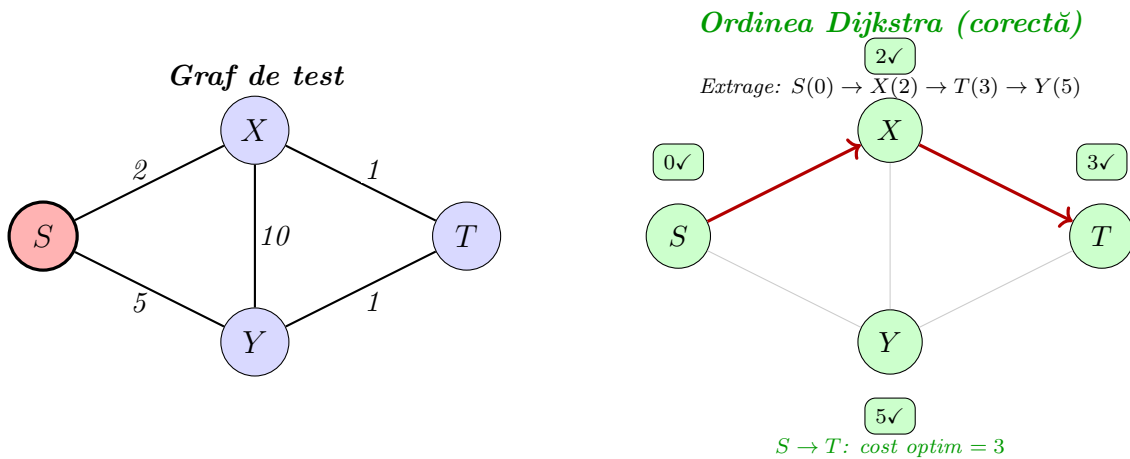
Arborele drumurilor scurte din A

Drumurile optime reconstituite (urmărind π de la destinație spre A):

- $A \rightarrow B$: cost **7** — direct pe muchia A-B
- $A \rightarrow C$: cost **9** — direct pe muchia A-C
- $A \rightarrow F$: cost **11** — $A \rightarrow C \rightarrow F$ (muchia directă A-F cu cost 14 este mai lungă!)
- $A \rightarrow D$: cost **17** — $A \rightarrow B \rightarrow D$
- $A \rightarrow E$: cost **20** — $A \rightarrow B \rightarrow D \rightarrow E$

5 Vizualizarea detaliată a coeziunii algoritmului

Exemplu 5.1. *De ce contează ordinea extragerii?* Ilustrăm cu un contraexemplu că extragerea în ordine greșită ar da rezultate incorecte.



Dijkstra extrage X (cu $d = 2$) înaintea lui Y (cu $d = 5$), și astfel descoperă drumul $S \rightarrow X \rightarrow T$ cu cost 3, înainte ca Y să poată "influența" T . Dacă am fi extras Y primul, am fi găsit $d[T] = 6$ și apoi X l-ar fi corectat la 3 — funcționează tot, dar ordinea Dijkstra este cea mai eficientă.

6 Implementare Python

```

1 import heapq
2
3 def dijkstra(graf, sursa):
4     """
5     Algoritmul lui Dijkstra cu coada de prioritati (min-heap).
6
7     Parametri:
8     graf    -- dictionar: {varf: [(vecin, cost), ...]}
9     sursa   -- varful de start
10
11     Returneaza:
12     d       -- dictionar cu distantele minime de la sursa
13     pi      -- dictionar cu predecesorii pe drumul optim
14     """
15     # Initializare
16     d = {v: float('inf') for v in graf}
17     pi = {v: None for v in graf}
18     d[sursa] = 0
19
20     # Coada de prioritati: (distanța_curentă, varf)
21     coada = [(0, sursa)]
22
23     while coada:
24         dist_u, u = heapq.heappop(coada) # Extrage minimul

```



```

25     # Daca am gasit deja o cale mai buna, ignoram
26     if dist_u > d[u]:
27         continue
28
29
30     # Relaxam toti vecinii lui u
31     for v, w in graf[u]:
32         if d[u] + w < d[v]:                # Relaxare reusita
33             d[v] = d[u] + w
34             pi[v] = u
35             heapq.heappush(coada, (d[v], v))
36
37     return d, pi
38
39
40 def reconstruieste_drum(pi, sursa, destinatie):
41     """
42     Reconstruieste drumul optim de la sursa la destinatie
43     folosind vectorul de predecesori pi.
44     """
45     drum = []
46     nod = destinatie
47     while nod is not None:
48         drum.append(nod)
49         nod = pi[nod]
50     drum.reverse()
51
52     # Verificam ca drumul chiar pleaca din sursa
53     if drum[0] != sursa:
54         return None # Nu exista drum
55     return drum

```

Exemplu 6.1. Testarea implementarii pe graful din Exemplul 4.1:

```

1  # Graful din exemplul pas-cu-pas
2  graf = {
3      'A': [('B', 7), ('C', 9), ('F', 14)],
4      'B': [('A', 7), ('C', 15), ('D', 10)],
5      'C': [('A', 9), ('B', 15), ('E', 11), ('F', 2)],
6      'D': [('B', 10), ('E', 6), ('F', 9)],
7      'E': [('C', 11), ('D', 6)],
8      'F': [('A', 14), ('C', 2), ('D', 9)]
9  }
10
11  d, pi = dijkstra(graf, 'A')
12
13  print("Distanțe minime din A:")
14  for varf in sorted(d):
15      drum = reconstruieste_drum(pi, 'A', varf)
16      print(f" A -> {varf}: cost {d[varf]:3d} drum: {' -> '.join(drum)}")
17

```

```

18 # Output:
19 # A -> A: cost    0   drum: A
20 # A -> B: cost    7   drum: A -> B
21 # A -> C: cost    9   drum: A -> C
22 # A -> D: cost   17   drum: A -> B -> D
23 # A -> E: cost   20   drum: A -> B -> D -> E
24 # A -> F: cost   11   drum: A -> C -> F

```

Exemplu 6.2. Analiza complexității implementării

```

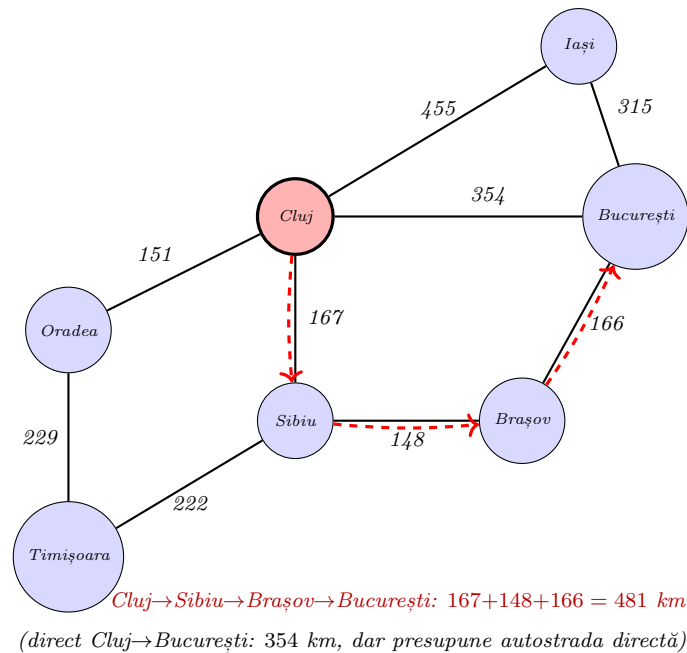
1 # Complexitate:  $O((|V| + |E|) * \log|V|)$ 
2 # - Fiecare varf e extras din heap o data:  $O(|V| * \log|V|)$ 
3 # - Fiecare muchie genereaza cel mult o  $O(|E| * \log|V|)$ 
4 #   insertie in heap
5 # Total:  $O((|V| + |E|) * \log|V|)$ 
6
7 # Pentru grafuri dense ( $|E| \sim |V|^2$ ):
8 #    $O(|V|^2 * \log|V|)$  cu heap
9 #    $O(|V|^2)$  cu matrice de adiacenta (varianta simpla)
10
11 # Demonstratie pe grafuri de dimensiuni diferite:
12 import time, random
13
14 def genereaza_graf_aleator(n, m, cost_max=100):
15     """Genereaza un graf aleator cu n varfuri si m muchii."""
16     graf = {i: [] for i in range(n)}
17     for _ in range(m):
18         u = random.randint(0, n-1)
19         v = random.randint(0, n-1)
20         w = random.randint(1, cost_max)
21         if u != v:
22             graf[u].append((v, w))
23             graf[v].append((u, w))
24     return graf
25
26 for n, m in [(100, 500), (1000, 5000), (10000, 50000)]:
27     g = genereaza_graf_aleator(n, m)
28     start = time.time()
29     dijkstra(g, 0)
30     elapsed = time.time() - start
31     print(f"n={n:6d}, m={m:6d}: {elapsed*1000:.2f} ms")

```

7 Aplicații practice

Exemplu 7.1. Rute de transport între orașe românești

Găsim drumul cel mai scurt (în km) între Cluj-Napoca și celelalte orașe principale.



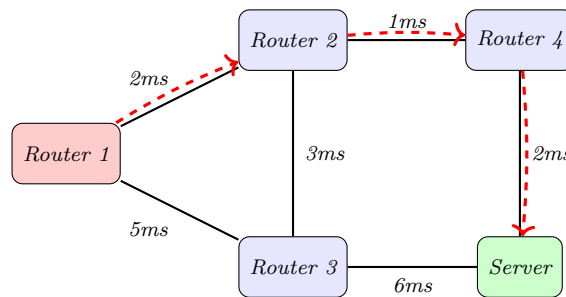
```

1 orase = {
2     'Cluj':      [('Oradea', 151), ('Sibiu', 167), ('Iasi', 455)
3                 ,
4                 ('Bucuresti', 354)],
5     'Oradea':    [('Cluj', 151),   ('Timisoara', 229)],
6     'Timisoara': [('Oradea', 229), ('Sibiu', 222)],
7     'Sibiu':     [('Cluj', 167),   ('Timisoara', 222), ('Brasov',
8                 , 148)],
9     'Brasov':    [('Sibiu', 148),  ('Bucuresti', 166)],
10    'Bucuresti':  [('Brasov', 166), ('Cluj', 354),      ('Iasi',
11                , 315)],
12    'Iasi':       [('Cluj', 455),   ('Bucuresti', 315)]
13 }
14
15 d, pi = dijkstra(orase, 'Cluj')
16
17 print("Distanțe minime din Cluj-Napoca:")
18 for oras in ['Oradea', 'Timisoara', 'Sibiu', 'Brasov', 'Bucuresti', 'Iasi']:
19     drum = reconstruieste_drum(pi, 'Cluj', oras)
20     print(f"    -> {oras:12s}: {d[oras]:4d} km ({' -> '.join(drum)}")
21
22 # Output:
23 # -> Oradea      : 151 km (Cluj -> Oradea)
24 # -> Timisoara   : 380 km (Cluj -> Oradea -> Timisoara)
25 # -> Sibiu       : 167 km (Cluj -> Sibiu)
26 # -> Brasov      : 315 km (Cluj -> Sibiu -> Brasov)
27 # -> Bucuresti   : 354 km (Cluj -> Bucuresti)
28 # -> Iasi        : 455 km (Cluj -> Iasi)

```

Exemplu 7.2. Rețea de calculatoare — latență minimă

Într-o rețea, ponderile reprezintă latența (ms) între noduri. Vrem să găsim ruta cu latență minimă.



Rută optimă: $R1 \rightarrow R2 \rightarrow R4 \rightarrow \text{Server}$, latență = $2 + 1 + 2 = 5$ ms

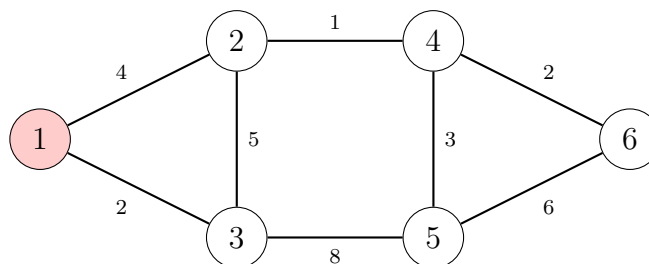
```

1  retea = {
2      'R1': [('R2', 2), ('R3', 5)],
3      'R2': [('R1', 2), ('R4', 1), ('R3', 3)],
4      'R3': [('R1', 5), ('R2', 3), ('Server', 6)],
5      'R4': [('R2', 1), ('Server', 2)],
6      'Server': [('R3', 6), ('R4', 2)]
7  }
8
9  d, pi = dijkstra(retea, 'R1')
10 drum = reconstruieste_drum(pi, 'R1', 'Server')
11 print(f"Ruta minima: {' -> '.join(drum)}")
12 print(f"Latenta totala: {d['Server']} ms")
13 # Output:
14 # Ruta minima: R1 -> R2 -> R4 -> Server
15 # Latenta totala: 5 ms

```

8 Exerciții propuse

1. Aplicați manual algoritmul Dijkstra pornind din vârful 1 pe graful de mai jos. Completați tabelul de urmărire și desenați arborele drumurilor scurte.



2. Rulați funcția `dijkstra` din seminar pe graful de la exercițiul 1 și verificați că obțineți aceleași rezultate ca aplicarea manuală.

3. Scrieți o funcție `dijkstra_k_drumuri(graf, sursa, dest, k)` care returnează cele mai scurte k drumuri distincte de la `sursa` la `dest` (cu costuri posibil egale). *Indiciu:* mențineți un contor de vizite per vârf.
4. Modificați implementarea Python pentru a returna și **numărul de drumuri optime** (nu doar un drum, ci câte drumuri distincte au costul minim) de la sursă la fiecare vârf.
5. O firmă de curierat are depozite în București, Cluj și Timișoara. Folosind graful orașelor din Exemplul 6.1, determinați care depozit are acces cel mai rapid (suma minimă a distanțelor optime) la toate celelalte orașe din graf. *Indiciu:* rulați Dijkstra din fiecare depozit și comparați sumele.